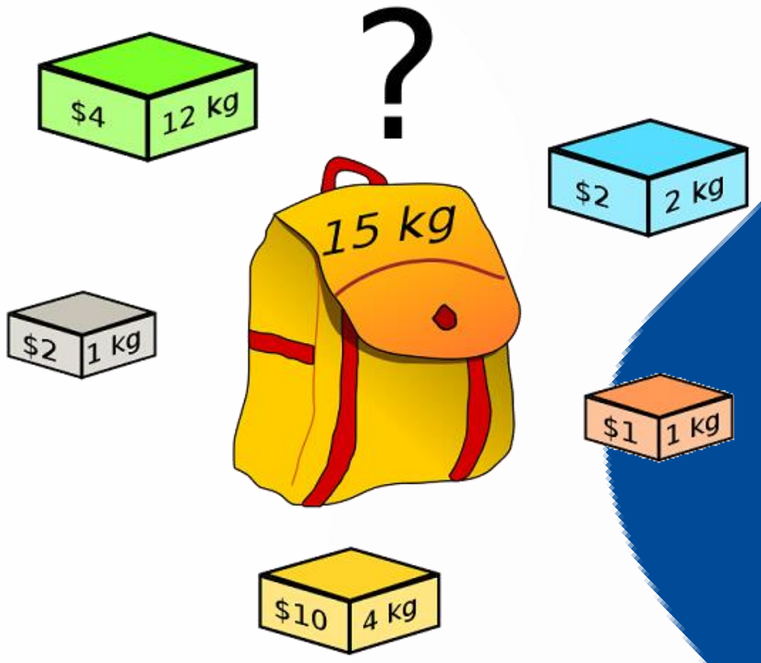


03.a

0/1 Knapsack Problem

0/1 Knapsack Problem

The 0/1 knapsack problem means that the items are either completely or no items are filled in a knapsack.

n=4

m=8

P_i	1	2	5	6
Wt_i	2	3	4	5
	Mirror	Silver nugget	Painting	Vase

n is number of items

m is the capacity of knapsack

0/1 Knapsack Problem

n=4

m=8

P_i	1	2	5	6
Wt_i	2	3	4	5

x_i = {1, 0, 0, 1}
{0, 0, 0, 1}
{0, 1, 0, 1}
{1, 0, 1, 0}
.
.
.
.
.

- 1 denotes that the item is completely picked and 0 means that no item is picked.
- Since there are 4 items so possible combinations will be: **2^4 = 16;** So. There are 16 possible combinations that can be made by using the above problem.
- Once all the combinations are made, we have to select the combination that provides the maximum profit.
- If there are n items so possible combinations will be: **2^n,**

So. There are 2^n possible combinations that can be made by using the above problem.

0/1 Knapsack Problem

n=4

m=8

P_i	1	2	5	6
Wt_i	2	3	4	5

x_i = {1, 0, 0, 1}
{0, 0, 0, 1}
{0, 1, 0, 1}
{1, 0, 1, 0}
.
.
.
.
.

- We can solve such type of problem using **dynamic programming approach**.

- In dynamic programming approach, the **complicated problem is divided into sub-problems**, then we find the solution of a sub-problem and the solution of the sub-problem will be used to find the solution of a large complex problem

Algorithm of 0/1 Knapsack Problem

n=4

m=8

P_i	1	2	5	6
W_i	2	3	4	5

Step- 1: We create a matrix/Table as shown as below (Table is named as T)

P_i	w_i	i \ w	0	1	2	3	4	5	6	7	8
		0									
1	2	1									
2	3	2									
5	4	3									
6	5	4									

- In this matrix, columns represent the weight, i.e., 8. The rows represent the profits and weights of items.
- Here we have not taken the weight 8 directly, problem is divided into sub-problems, i.e., 0, 1, 2, 3, 4, 5, 6, 7, 8.
- The solution of the sub-problems would be saved in the cells and answer to the problem would be stored in the final cell.
- First, we write the weights in the **ascending order** and profits according to their weights as shown.

Algorithm of 0/1 Knapsack Problem

n=4

m=8

P_i	1	2	5	6
W_i	2	3	4	5

Here, T(i , w) = maximum value of the selected items

<u>Step-02:</u> Start filling the table row wise top to bottom from left to right.

Use the following formula-

T (i , w) = max { T [i-1] [w] , P_i + T[i-1][w– wt_i] } // Value of T(i, w) is max profit

		w / i	0	1	2	3	4	5	6	7	8
P_i	w_i	0									
1	2	1									
2	3	2									
5	4	3									
6	5	4									

- This step leads to completely filling the table.

- Then, value of the last box represents the maximum possible profit that can be put into the knapsack.

Algorithm of 0/1 Knapsack Problem

n=4

m=8

P_i	1	2	5	6
W_i	2	3	4	5

Step-03:

To identify the items that must be put into the knapsack to obtain that maximum profit, Consider the last column of the table.

1) Start scanning the entries from bottom to top.

2) On encountering an entry whose value is not same as the value stored in the entry immediately above it, mark the row label of that entry.

3) After all the entries are scanned, the marked labels represent the items that must be put into the knapsack.

Exercise of 0/1 Knapsack Problem

n=4

m=8

P_i	1	2	5	6
Wt_i	2	3	4	5

Step-02: Start filling the table row wise top to bottom from left to right.

Use the following formula-

$$T(i, w) = \max\{T[i-1][w], P_i + T[i-1][w - wt_i]\}$$

P_i	w_i
1	2
2	3
5	4
6	5

i \ w	0	1	2	3	4	5	6	7	8
0									
1									
2									
3									
4									

Or Pseudo Code

```
Knapsack(p,wt,m,n)
  for i=0 to n
    for w=0 to m
      if i==0 || w==0
        T[i][w]=0
      else
        if wt[i]<=w
          T[i][w]=max(p[i]+T[i-1][w-wt[i]], T[i-1][w])
      else
          T[i][w]=T[i-1][w]
```

Solving 0/1 Knapsack Problem

Step-02: Start filling the table row wise top to bottom from left to right.

Use the following formula-

$$T(i, w) = \max\{ T[i-1][w], P_i + T[i-1][w - wt_i] \}$$

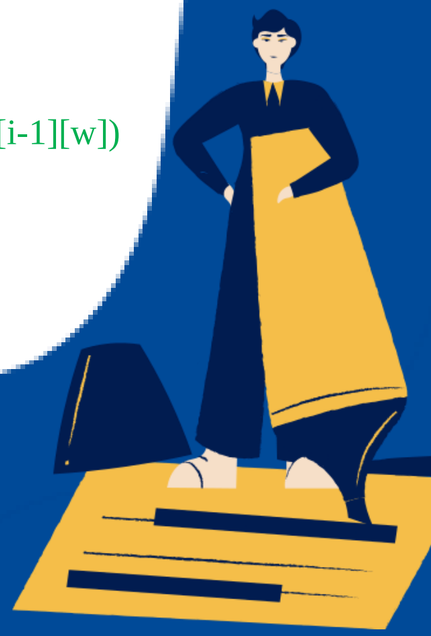
For i=0

w= 0 w= 1 w= 2 w= 8

T[0][0]=0 T[0][1]=0 T[0][2]=0 …… T[0][8]=0

		i \ w	0	1	2	3	4	5	6	7	8
P_i	w_i	0	0	0	0	0	0	0	0	0	0
1	2	1									
2	3	2									
5	4	3									
6	5	4									

```
Knapsack(p,wt,m,n)
  for i=0 to n
    for w=0 to m
      if i==0 || w==0
        T[i][w]=0
    else if wt[i]<=w
      T[i][w]=max(p[i]+T[i-1][w-wt[i]], T[i-1][w])
    else
      T[i][w]=T[i-1][w]
```



Solving 0/1 Knapsack Problem

Step-02: Start filling the table row wise top to bottom from left to right.

Use the following formula-

$$T(i, w) = \max\{ T[i-1][w], P_i + T[i-1][w - wt_i] \}$$

For i=1

w= 0

T[1][0]=0

w= 1

if 2 < 1

else

T[1][1] = T[1][1]= 0

w= 2

if 2 =2

T[1][2] = max (1+ T[0][0], T[0][2])

= max(1 + 0 , 0)

= 1

w= 3

if 2 <3

T[1][3] = max (1+ T[0][1], T[0][3])

= max(1 + 0 , 0)

= 1

P_i	w_i	i \ w	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2									
5	4	3									
6	5	4									

w= 4

if 2 <4

T[1][4] = max (1+ T[0][2], T[0][4])

= max(1 + 0 , 0)

= 1

Continue……..

Or

Pseudo Code

```
Knapsack(p,wt,m,n)
  for i=0 to n
    for w=0 to m
      if i==0 || w==0
        T[i][w]=0
      else if wt[i]<=w
        T[i][w]=max(p[i]+T[i-1][w-wt[i]], T[i-1][w])
      else
        T[i][w]=T[i-1][w]
```

Solving 0/1 Knapsack Problem

```
Knapsack(p,wt,m,n)    for i=0 to n
    for w=0 to m
      if i==0 || w==0
        T[i][w]=0
      else if wt[i]<=w
        T[i][w]=max(p[i]+T[i-1][w-wt[i]], T[i-1][w])
       else
        T[i][w]=T[i-1][w]
```

Step-02: Start filling the table row wise top to bottom from left to right.

Use the following formula-

$$T(i, w) = \max\{ T[i-1][w], P_i + T[i-1][w - wt_i] \}$$

For i=2

w= 0

T[2][0]=0

w= 1

if 3 < 1

else

T[2][1] = T[1][1]= 0

w= 2

if 3 < 2

else

T[2][2] = T[1][2]= 1

w= 3

if 3 =3

T[2][3] = max (2+ T[1][0], T[1][3])

= max(2 + 0 , 1)

= 2

i=2, w= 4

if 3 <4

T[2][4] = max (2+ T[1][1], T[1][4])

= max(2 + 0 , 1)

= 2

i=2, w= 5

if 3 <5

T[2][5] = max (2+ T[1][2], T[1][5])

= max(2 + 1 , 1)

= 3

i=2, w= 6

if 3 <6

T[2][6] = max (2+ T[1][3], T[1][6])

= max(2 + 1 , 1)

= 3

P_i	w_i	i \ w	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3									
6	5	4									

Continue……..

Solving 0/1 Knapsack Problem

```
Knapsack(p,wt,m,n)
  for i=0 to n
    for w=0 to m
      if i==0 || w==0
        T[i][w]=0
      else if wt[i]<=w
        T[i][w]=max(p[i]+T[i-1][w-wt[i]], T[i-1][w])
       else
      T[i][w]=T[i-1][w]
```

Step-02: Start filling the table row wise top to bottom from left to right.

Use the following formula-

$$T(i, w) = \max\{ T[i-1][w], P_i + T[i-1][w - wt_i] \}$$

For i=3

w= 0

T[3][0]=0

w= 1
if 4 < 1
else
T[3][1] = T[2][1]= 0

w= 2
if 4 < 2
else
T[3][2] = T[2][2]= 1

w= 3
if 4 < 3
else
T[3][3] = T[2][3]= 2

w= 4
if 4 =4
T[3][4] = max (5+ T[2][0], T[2][4])
= max(5 + 0 , 2)
= 5

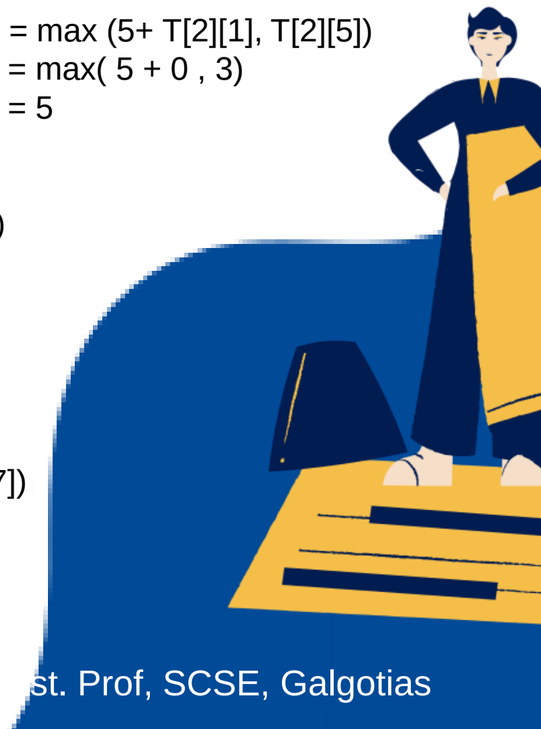
i=3, w= 5
if 4 <5
T[3][5] = max (5+ T[2][1], T[2][5])
= max(5 + 0 , 3)
= 5

i=3, w= 6
if 4 <6
T[3][6] = max (5+ T[2][2], T[2][6])
= max(5 + 1 , 3)
= 6

i=3, w= 7
if 4 <7
T[3][7] = max (5+ T[2][3], T[2][7])
= max(5 + 2 , 3)
= 7

Continue……..

P_i	w_i	i \ w	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4									



Solving 0/1 Knapsack Problem

Step-02: Start filling the table row wise top to bottom from left to right.

Use the following formula-

$$T(i, w) = \max \{ T[i-1][w], P_i + T[i-1][w - wt_i] \}$$

```
Knapsack(p,wt,m,n)
  for i=0 to n
    for w=0 to m
      if i==0 || w==0
        T[i][w]=0
      else if wt[i]<=w
        T[i][w]=max(p[i]+T[i-1][w-wt[i]], T[i-1][w])
       else
      T[i][w]=T[i-1][w]
```

For i=4

w= 0

T[4][0]=0

w= 1
if 5 < 1
else
T[4][1] = T[3][1]= 0

w= 2
if 5 < 2
else
T[4][2] = T[3][2]= 1

w= 3
if 5 < 3
else
T[4][3] = T[3][3]= 2

w= 4
if 5 < 4
else
T[4][4] = T[3][4]= 5

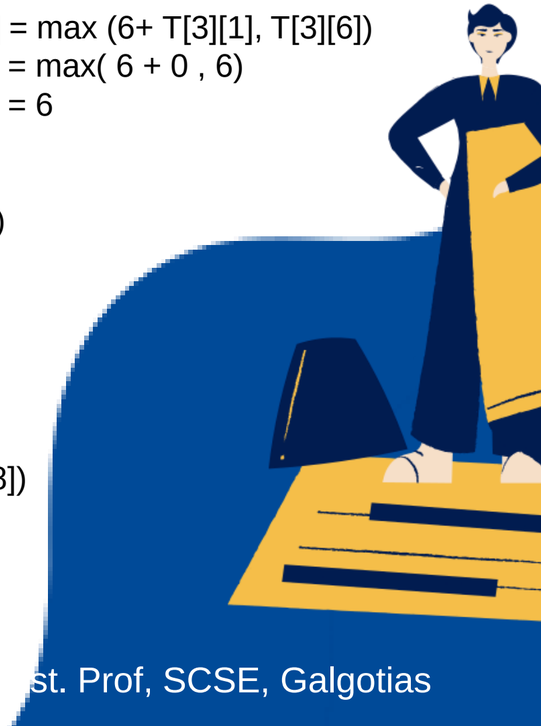
w= 5
if 5 =5
T[4][5] = max (6+ T[3][0], T[3][5])
= max(6 + 0 , 5)
= 6

i=4, w= 6
if 5 <6
T[4][6] = max (6+ T[3][1], T[3][6])
= max(6 + 0 , 6)
= 6

i=4, w= 7
if 5 <7
T[4][7] = max (6+ T[3][2], T[3][7])
= max(6 + 1 , 7)
= 7

i=4, w= 8
if 5 <8
T[4][8] = max (6+ T[3][3], T[3][8])
= max(6 + 2 , 7)
= max (8, 7)
= 8

P_i	w_i	i \ w	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8



Solving 0/1 Knapsack Problem

Value of T(i, w) is max profit , here T[4][8]= 8 is the maximum profit of Knapsack

P_i	w_i	i \ w	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

Step-03: To identify the items that must be put into the knapsack to obtain that maximum profit, Consider the last column of the table.

1) Start scanning the entries from bottom to top.
2) On encountering an entry whose value is not same as the value stored in the entry immediately above it, mark the row label of that entry.
3) After all the entries are scanned, the marked labels represent the items that must be put into the knapsack.

Solving 0/1 Knapsack Problem

Value of T(i, w) is max profit , here T[4][8]= 8 is the maximum profit

P_i	w_i	i \ w	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

<u>Step-03:</u>

To identify the items that must be put into the knapsack to obtain that maximum profit,

Consider the last column of the table.

1) **Start scanning the entries from bottom to top.**
2) **On encountering an entry whose value is not same as the value stored in the entry immediately above it, mark the row label of that entry.**
3) **After all the entries are scanned, the marked labels represent the items that must be put into the knapsack.**

<u>Step-03:</u>

```
while (i > 0 && w>0)
   if (T[i][w] == T[i-1][w]
      Not select the current item
      i--
   else
      select the item i
      i--;
      w= w-wt[i]
```

Solving 0/1 Knapsack Problem

Value of T(i, w) is max profit , here T[4][8]= 8 is the maximum profit

P_i	w_i	i \ w	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

Step-03:

To identify the items that must be put into the knapsack to obtain that maximum profit,

Consider the last column of the table.

1) **Start scanning the entries from bottom to top.**
2) **On encountering an entry whose value is not same as the value stored in the entry immediately above it, mark the row label of that entry.**
3) **After all the entries are scanned, the marked labels represent the items that must be put into the knapsack.**

Step-03:

```
while (i > 0 && w>0)
    if (T[i][w] == T[i-1][w]
        Not select the current item
        i--
    else
        select the item i
        i--;
        w= w-wt[i]
```

Solving 0/1 Knapsack Problem

Value of T(i, w) is max profit , here T[4][8]= 8 is the maximum profit

P_i	w_i	i \ w	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

Step-03:

To identify the items that must be put into the knapsack to obtain that maximum profit,

Consider the last column of the table.

1) **Start scanning the entries from bottom to top.**
2) **On encountering an entry whose value is not same as the value stored in the entry immediately above it, mark the row label of that entry.**
3) **After all the entries are scanned, the marked labels represent the items that must be put into the knapsack.**

Step-03:

```
while (i > 0 && w>0)
    if (T[i][w] == T[i-1][w]
        Not select the current item
        i--
    else
        select the item i
        i--;
        w= w-wt[i]
```

Solving 0/1 Knapsack Problem

Value of T(i, w) is max profit , here T[4][8]= 8 is the maximum profit

P_i	w_i	i \ w	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

<u>Item Selected = { 0, 1, 0, 1}</u>

Time Complexity of 0/1 Knapsack Problem

Finding Maximum Profit

```
Knapsack(p,wt,m,n)
    for i=0 to n
        for w=0 to m
            if i==0 || w==0
                T[i][w]=0
            else if wt[i]<=w
                T[i][w]=max(p[i]+T[i-1][w-wt[i]], T[i-1][w])
            else
                T[i][w]=T[i-1][w]
```

n*m

Finding List of Items

```
while (i > 0 && w>0)
    if (T[i][w] == T[i-1][w]
        Not select the current item
        i--
    else
        select the item i
        i--;
        w= w-wt[i]
```

n

Total Time= n*m + n
= **O(nm)**